



Rapport Technique de Projet

Solution professionnelle de monitoring et gestion des fuites d'eau

1. Introduction & Contexte du Projet

WaterAlert est né d'un constat simple mais préoccupant : les fuites d'eau urbaines sont souvent signalées tardivement, traitées de façon désorganisée, et leurs données restent inexploitées. Ce projet vise à combler ce vide en offrant une plateforme complète qui connecte les citoyens, les opérateurs de terrain et les décideurs dans un écosystème cohérent et intelligent.

La solution repose sur trois piliers complémentaires : un bot Telegram pour faciliter le signalement citoyen, un tableau de bord analytique propulsé par l'IA pour la prise de décision, et une API REST pour l'interopérabilité avec d'autres systèmes.

2. Objectifs du Projet

2.1 Objectifs fonctionnels

- Permettre à tout citoyen de signaler une fuite d'eau rapidement via Telegram, sans compétences techniques particulières.
- Analyser automatiquement la gravité d'une fuite grâce à la vision artificielle (Google Gemini).
- Visualiser les données de signalement en temps réel sur un tableau de bord interactif.
- Géolocaliser les fuites et identifier les zones critiques via une carte de chaleur (heatmap).
- Exposer les données via une API REST pour permettre l'intégration avec d'autres outils.

2.2 Objectifs non-fonctionnels

- Déploiement simple et reproductible grâce à un environnement Python venv documenté.
- Sécurité des credentials (tokens Telegram, clés API) via variables d'environnement (.env).
- Mode de simulation automatique en l'absence de clé IA, garantissant la continuité du service.
- Architecture modulaire permettant l'évolution indépendante de chaque composant.

3. Architecture du Système

WaterAlert adopte une architecture en couches clairement séparées, facilitant la maintenabilité et l'évolutivité du projet. Chaque composant peut être démarré et testé indépendamment.

Composant	Rôle	Technologie
Bot Telegram	Interface citoyenne de signalement	python-telegram-bot
Dashboard	Visualisation & analyse des données	Streamlit + Plotly
API REST	Exposition des données pour tiers	FastAPI

Base de données	Persistance des signalements	SQLite
IA / Vision	Détection automatique de gravité	Google Gemini Vision
Géocoding	Localisation géographique	geopy
Export	Génération de rapports	CSV / PDF

3.1 Structure des fichiers

Le projet est organisé selon le schéma suivant :

- data/ — Base de données SQLite stockant l'ensemble des signalements.
- src/bot/ — Logique du bot Telegram (commandes, handlers, workflow de signalement).
- src/dashboard/ — Interface Streamlit avec ses assets CSS et composants graphiques.
- src/api/ — Endpoints FastAPI pour l'accès programmatique aux données.
- src/database/ — Couche d'abstraction de la base de données (historique et tracking).
- src/utlis/ — Utilitaires transversaux : IA (Gemini), géocoding (geopy), génération PDF.
- streamlit_app.py — Point d'entrée principal pour le déploiement.
- verify_setup.py — Script de diagnostic pour valider l'installation complète.

4. Construction du Projet — Du Début à la Fin

Étape 1 — Définition du besoin et choix de l'architecture

La première phase a consisté à définir précisément le périmètre du projet. La décision d'utiliser Telegram comme canal de signalement s'est imposée pour sa simplicité d'accès et son API gratuite et robuste. Streamlit a été retenu pour le dashboard en raison de sa courbe d'apprentissage faible et de son intégration native avec Python. FastAPI a été choisi pour l'API REST pour ses performances et la génération automatique de documentation.

Étape 2 — Mise en place de la base de données

SQLite a été choisi pour sa simplicité de déploiement (aucun serveur nécessaire). La couche database a été construite en premier pour servir de fondation à tous les autres composants. Elle gère deux aspects essentiels : l'historique des signalements et le tracking de l'état de traitement de chaque fuite.

Étape 3 — Développement du Bot Telegram

Le bot constitue le point d'entrée principal des données. Son développement a suivi un workflow structuré : l'utilisateur décrit le problème, envoie une photo, indique sa localisation, et le bot orchestre la validation et l'enregistrement du signalement. Des commandes standard (/start, /help, /status, /about, /privacy, /contact) ont été implémentées pour offrir une expérience professionnelle. Des indicateurs de saisie ('typing...') ont été ajoutés pour améliorer la perception de réactivité.

Étape 4 — Intégration de l'Intelligence Artificielle

L'intégration de Google Gemini Vision représente l'une des étapes les plus complexes du projet. Un système de 'détection hybride' a été mis en place : l'IA analyse la photo soumise par le citoyen pour estimer la gravité de la fuite, et ce résultat est combiné avec l'évaluation subjective du citoyen. Un mode simulation a également été développé pour permettre au système de fonctionner même sans clé API valide, garantissant la robustesse de la solution.

Étape 5 — Développement du tableau de bord

Le dashboard Streamlit a été construit autour de KPIs clés : nombre de signalements, répartition par gravité, temps moyen de traitement, tendances temporelles. Plotly a été utilisé pour les graphiques interactifs. La carte de chaleur permet d'identifier visuellement les zones géographiques les plus touchées, pondérée par la sévérité des fuites. Des fonctionnalités d'export en CSV et en PDF ont également été développées pour répondre aux besoins de reporting.

Étape 6 — Développement de l'API REST

FastAPI expose les données de signalement à travers un endpoint structuré. Cette couche permet à des systèmes externes (applications mobiles, outils de gestion municipale, etc.) d'accéder aux données de WaterAlert de façon standardisée.

Étape 7 — Tests, validation et documentation

Un script dédié (verify_setup.py) a été développé pour diagnostiquer l'état de l'installation : vérification des dépendances, validité des tokens, connexion à la base de données. Le fichier README.md centralise toutes les instructions d'installation et d'utilisation. La gestion de la sécurité (fichier .gitignore pour les credentials) a été intégrée dès le début du projet.

5. Technologies Utilisées

Technologie	Version/Type	Usage dans WaterAlert
Python	3.x	Langage principal du projet
python-telegram-bot	Library	Bot Telegram (signalement citoyen)
Streamlit	Framework	Dashboard interactif (frontend)
Plotly	Library	Graphiques et visualisations interactives
FastAPI	Framework	API REST (exposition des données)
SQLite	Base de données	Persistance locale des signalements
Google Gemini Vision	API IA	Analyse visuelle des photos de fuites
geopy	Library	Géocoding et localisation géographique
python-dotenv	Library	Gestion des variables d'environnement
ReportLab / FPDF	Library	Génération de rapports PDF
Git + .gitignore	Outil	Versioning et sécurité des credentials

6. Difficultés Rencontrées

6.1 Gestion des environnements Python

La gestion de l'environnement virtuel (venv) a posé des problèmes de compatibilité sur certaines configurations Windows, notamment liés aux politiques d'exécution PowerShell bloquant les scripts d'activation. Une documentation spécifique et des alternatives (installation globale des packages) ont dû être prévues pour contourner ces obstacles.

6.2 Intégration de Google Gemini Vision

L'API Gemini Vision nécessite une gestion fine des formats d'images, des timeouts réseau et des réponses partielles ou ambiguës du modèle. La conception du mode simulation a demandé un effort particulier pour garantir une dégradation gracieuse du service sans impacter l'expérience utilisateur.

6.3 Géocoding et précision des localisations

La bibliothèque geopy est dépendante de services tiers (Nominatim, Google Maps) qui imposent des limites de taux d'appels (rate limiting). En milieu urbain dense, la résolution d'adresses peut être imprécise. Des stratégies de cache et de fallback ont dû être envisagées pour garantir la fiabilité de la fonctionnalité cartographique.

6.4 Synchronisation des composants asynchrones

Le bot Telegram utilise un modèle asynchrone (asyncio) tandis que Streamlit fonctionne en mode synchrone. La coexistence de ces deux paradigmes dans le même projet a nécessité une architecture soignée pour éviter les conflits de boucles d'événements, notamment lors des accès concurrents à la base de données SQLite.

6.5 Génération de rapports PDF

La génération de documents PDF professionnels directement depuis Python présente des défis de mise en page, notamment pour les tableaux contenant des données variables et les caractères spéciaux (accents, symboles). Des ajustements itératifs ont été nécessaires pour obtenir des exports propres et lisibles.

6.6 Taille du projet et déploiement

Avec un poids de 120 Mo, le projet inclut vraisemblablement des assets lourds (images, données de test, environnement venv intégré). Cela pose des questions pour le déploiement en cloud ou le partage du code, et anticipe un besoin de rationalisation des dépendances.

7. Points Forts du Projet

- Architecture modulaire : chaque composant (bot, dashboard, API) peut être lancé et maintenu indépendamment.

- Expérience utilisateur soignée : indicateurs de frappe, menu de commandes cliquable, guide intégré.
- Détection hybride IA : combinaison de l'input citoyen et de l'analyse Gemini Vision pour plus de fiabilité.
- Mode simulation : continuité de service garantie même sans clé API IA valide.
- Sécurité intégrée dès le départ : gestion des secrets via `.env` et `.gitignore`.
- Script de diagnostic (`verify_setup.py`) facilitant l'installation et le débogage.
- Exportation multi-format (CSV et PDF) pour répondre aux besoins de reporting.

8. Pistes d'Amélioration pour le Futur

8.1 Infrastructure et déploiement

- Migrer de SQLite vers PostgreSQL pour supporter un usage multi-utilisateurs en production.
- Containeriser le projet avec Docker (docker-compose) pour simplifier le déploiement cloud.
- Mettre en place une pipeline CI/CD (GitHub Actions) pour automatiser les tests et déploiements.
- Déployer le dashboard sur Streamlit Community Cloud ou Heroku pour un accès public.

8.2 Intelligence Artificielle

- Entraîner un modèle de classification personnalisé sur des images de fuites d'eau pour réduire la dépendance à Gemini.
- Ajouter une prédiction du temps d'intervention basée sur l'historique des données (ML supervisé).
- Intégrer un système de détection d'anomalies pour identifier les zones à risque avant même les signalements.

8.3 Fonctionnalités utilisateur

- Développer une interface web (React ou Vue.js) en complément du bot Telegram pour les utilisateurs non familiers avec l'application.
- Ajouter un système de suivi en temps réel permettant au citoyen de connaître l'avancement du traitement de son signalement.
- Implémenter un système de notifications proactives (alertes par zone, résumés hebdomadaires).
- Ajouter le support multilingue pour les zones géographiques francophones, anglophones, arabophones.

8.4 Analyse et Business Intelligence

- Intégrer un moteur de reporting avancé (ex. Apache Superset) pour des analyses plus poussées.

- Ajouter des prévisions de charge saisonnières (modèles de séries temporelles).
- Développer un scoring de criticité pondéré par facteurs multiples (débit estimé, localisation, historique).

8.5 Sécurité et conformité

- Mettre en place une authentification utilisateur pour le dashboard (SSO, OAuth2).
- Implémenter une politique de rétention des données conforme au RGPD.
- Chiffrer les données sensibles stockées en base de données.
- Ajouter des logs d'audit pour tracer toutes les actions effectuées sur le système.

8.6 Performance

- Implémenter un système de cache (Redis) pour les requêtes fréquentes au dashboard.
- Optimiser la taille du projet en excluant le venv du repository et en utilisant un requirements.txt précis.
- Mettre en place un système de queue (Celery + Redis) pour traiter les analyses IA de façon asynchrone à fort volume.

9. Conclusion

WaterAlert constitue une réponse concrète et bien architecturée au problème de la gestion des fuites d'eau urbaines. En combinant un canal de signalement accessible (Telegram), une analyse intelligente (Google Gemini Vision), une visualisation analytique (Streamlit + Plotly) et une API ouverte (FastAPI), le projet couvre l'ensemble du cycle de vie d'un signalement — de la détection citoyenne à la prise de décision opérationnelle.

Les défis techniques rencontrés, notamment la gestion des environnements Python, l'intégration de l'IA et la synchronisation des composants asynchrones, ont été surmontés avec des approches pragmatiques comme le mode simulation et une architecture clairement modulaire. Ces choix techniques démontrent une maturité de conception au-delà d'un simple prototype.

Le projet dispose d'une base solide pour évoluer vers une solution de niveau production, notamment via la migration vers une base de données relationnelle robuste, la containerisation Docker, et l'enrichissement des capacités analytiques par le Machine Learning. WaterAlert a le potentiel de devenir un outil de référence pour la gestion intelligente des infrastructures hydrauliques urbaines.